04 - Intro to Assembly

Remember me?

REGISTERS

R0:0000000A	R8:00000000	R16:00000000	R24:00000000
R1:0000000A	R9:00000128	R17:00000000	R25:00000000
R2:00000022	R10:00000000	R18:00000000	R26:00000000
R3:00000037	R11:00000000	R19:00000000	BP:00000000
R4:00000037	R12:00000000	R20:00000000	LP:00000000
R5:00000000	R13:00000000	R21:00000000	SP:00000000
R6:00000000	R14:00000000	R22:00000000	XP:00000000
R7:00000000	R15:00000000	R23:00000000	R31:00000000

INSTRUCTIONS (SUPERVISOR MODE)

```
     028:77E50007          BEQ(R5,done)
     02C:80821800          ADD(R2,R3,R4)
     030:C1290004          ADDC(R9,4,R9)
     034:64890000          ST(R4,0,R9)
     038:8043F800          ADD(R3,R31,R2)
     03C:8064F800          ADD(R4,R31,R3)
     040:C0000001          ADDC(R0,1,R0)
     044:77FFFFF7          BR(loop)
PC ->048:00000000 done:    HALT()
     04C:00000000          HALT()
     050:00000000          HALT()
     054:00000000          HALT()
     058:00000000          HALT()
     05C:00000000          HALT()
     060:00000000          HALT()
     064:00000000          HALT()
```

STACK

```
     39C:00000000
     3A0:00000000
     3A4:00000000
     3A8:00000000
     3AC:00000000
     3B0:00000000
     3B4:00000000
     3B8:00000000
     3BC:00000000
     3C0:00000000
     3C4:00000000
     3C8:00000000
     3CC:00000000
     3D0:00000000
     3D4:00000000
     3D8:00000000
     3DC:00000000
     3E0:00000000
     3E4:00000000
     3E8:00000000
     3EC:00000000
     3F0:00000000
     3F4:00000000
     3F8:00000000
     3FC:00000000
SP ->000:603F0100
```

MEM[0x134]

```
100:0000000A
104:00000001
108:00000001
10C:00000002
110:00000003
114:00000005
118:00000008
11C:0000000D
120:00000015
124:00000022
128:00000037
12C:00000000
130:00000000
134:00000000
138:00000000
13C:00000000
140:00000000
144:00000000
148:00000000
14C:00000000
150:00000000
154:00000000
158:00000000
15C:00000000
160:00000000
164:00000000
```

GNU Assembler

- The default assembler for the GNU OS.
- Known by its executable name `as` (but we call it GAS).
  - Note: we won't use `as` directly (it's implicitly called by g++)
- Converts our assembly language program into a machine language program.
  - For this class, we shall use x86_64 machine language

CSCI 50 Slides on ML and Assembly Programming

How do we program a CPU?

High Level Language Program

z=x+y;

Compiler

Assembly Language Program

```
LD(R31,X,r1)
LD(R31,Y,r2)
ADD(r1,r2,r3)
ST(r3,Z,R31)
```

Assembler

Machine Language Program

```
0110 0000 0011 1111 0000 0010 0000 0000
0110 0000 0101 1111 0000 0010 0000 0100
1000 0000 0110 0001 0001 0000 0000 0000
0110 0100 0111 1111 0000 0010 0000 1000
```

Machine Interpretation

Control Signal Specification

ALUOP[0:3] <= InstReg[9:11] & MASK

DISCS

CSCI 50 Slides on ML and Assembly Programming

Instruction → 32 Bits

- `SUB ( R1, R2, R3 )`
- SUB's OPCODE = 0b100001
- RC = 3 = 0b00011
- RA = 1 = 0b00001
- RB = 2 = 0b00010
- Result: 0b 100001 00011 00001 00010 00000000000
  - 0x 84 61 10 00

DISCS

Order of arguments

- GAS uses *AT&T assembly syntax* by default
  - Has the same order of arguments as our Beta assembly.
  - The source parameters are always first, followed by the destination parameter.
- The other popular format is *Intel assembly syntax*
  - The destination is first, followed by the source.

AT&T vs Intel Syntax

- What does "`<op> A, B`" mean?

// AT&T: B ← B <op> A

```
movl $1, %eax
addl (%ebx, %ecx, 0x2), %eax
subq -20(%rbx), %rax
```

// Intel: A ← A <op> B

```
mov eax, 1
add eax, [ebx+ecx*2h]
sub rax, [rbx-20]
```

Beta vs. x86_64

- Recall Beta's instructions for register+register and register+literal(constant) ALU operations. How is x86_64 different?

5.1 ADD

Usage: ADD(Ra,Rb,Rc)

Opcode:

100000	Rc	Ra	Rb	*unused*

Operation: PC ← PC + 4

Reg[Rc] ← Reg[Ra] + Reg[Rb]

The contents of register Ra are added to the contents of register Rb and the 32-bit sum is written to Rc. This instruction computes no carry or overflow information. If desired, this can be computed through explicit compare instructions.

5.2 ADDC

Usage: ADDC(Ra,literal,Rc)

Opcode:

110000	Rc	Ra	literal

Operation: PC ← PC + 4

Reg[Rc] ← Reg[Ra] + SEXT(literal)

The contents of register Ra are added to *literal* and the 32-bit sum is written to Rc. This instruction computes no carry or overflow information. If desired, this can be computed through explicit compare instructions.

Generating the Assembly Code

Generating the Assembly Code

- Given the C++ sample code below:

```
// cppAssembly1.cpp
// Don't include headers or main() function!
int dummy( int a, int b ){
    int x = a * b;
    return x;
}
```

Generating the Assembly Code

- Recall: How do we usually compile our program?

```
g++ -o outputFile inputFile.cpp
```

- To generate assembly code instead, use:

```
g++ -S -o cppAssembly1.s cppAssembly1.cpp
```

- If you're on a non-Intel Mac, please add `-arch x86_64` so you can follow the discussion
- `-S` means "stop after compiling"
- As before, `-o` sets the output's filename. When generating assembly code, use the extension `.s`

Generating the Assembly Code

- Now we open the cppAssembly1.s file.
- Don't be overwhelmed!

cppAssembly1.s

```
	.file	"cppAssembly1.cpp"
	.text
	.globl	_Z5dummyii
	.type	_Z5dummyii, @function
_Z5dummyii:
.LFB0:
	.cfi_startproc
	endbr64
	pushq	%rbp
	.cfi_def_cfa_offset 16
	.cfi_offset 6, -16
	movq	%rsp, %rbp
	.cfi_def_cfa_register 6
	movl	%edi, -20(%rbp)
	movl	%esi, -24(%rbp)
	movl	-20(%rbp), %eax
	imull	-24(%rbp), %eax
	movl	%eax, -4(%rbp)
	movl	-4(%rbp), %eax
	popq	%rbp
	.cfi_def_cfa 7, 8
	ret
	.cfi_endproc
.LFE0:
	.size	_Z5dummyii, .-_Z5dummyii
	.ident	"GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0"
```

Generating the Assembly Code

- Focus on just the instruction lines of your "dummy" function, here → → →
  - You may not get the exact same instructions
  - For now, also ignore the `movl %edi` and `movl %esi` instructions
- Does this look familiar?

```
_Z5dummyii:

    pushq   %rbp

    movq    %rsp, %rbp

    movl    %edi, -20(%rbp)
    movl    %esi, -24(%rbp)
    movl    -20(%rbp), %eax
    imull   -24(%rbp), %eax
    movl    %eax, -4(%rbp)
    movl    -4(%rbp), %eax
    popq    %rbp

    ret
```